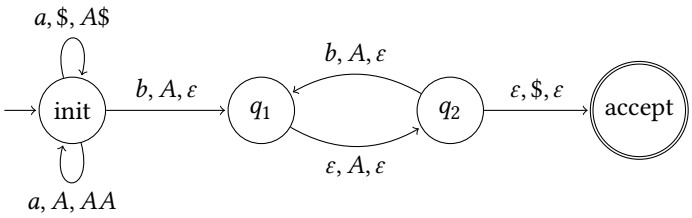
CS 2490 – Homework 8

Turn in by 2026/11/7 at 11:59pm

You **must** turn in your assignment as a zip file entitled: *lastname_firstname_hw8.zip*. It will contain a pdf file *lastname_firstname_hw8.pdf* with your solutions to the exercises, and a JSON file *lastname_firstname_hw8_ex2.json* for the PDA of Exercise 2.

Exercise 1. Consider the following PDA[1] $\mathscr{A}$:

[1] The stack alphabet is $\Gamma = \{A, \$\}$ and the initial stack symbol is \$.



(i) Does $\mathscr{A}$ satisfy the two restrictions of Session 28 that Ginsburg's method needs? If yes, say why. If no, transform it into an equivalent PDA that does.

(ii) Which triples (p, X, q) of Ginsburg's method are useful, and which ones are useless? Give a quick justification, using the meaning of the triples.[2]

[2] You may justify a whole group of triples at once, e.g., all the triples of the form (accept, X, q).

(iii) Construct a context-free grammar that generates the language recognized by $\mathscr{A}$, following Ginsburg's triples method. Trim the grammar as you go: skip every rule that contains a useless triple.

(iv) Rewrite your grammar with more readable names for the variables, and simplify it.

(v) What is the language generated by your grammar?

Exercise 2. In this exercise, we work over the alphabet $\Sigma = \{a, b, c\}$. Construct a PDA with exactly 3 states that recognizes the language

$$L_0 = \{a^n b^m c^{n+m} \mid n, m \geq 0\}.$$

(Simply draw the PDA, you don't have to justify that it indeed recognizes L_0.)

Include the JSON file of your PDA (produced with the web application) in your submission.

Exercise 3. In this exercise, we work over the alphabet $\Sigma = \{0, 1\}$. In Session 24, we saw that the grammar

$$G: \quad S \to 0S1 \mid 1S0 \mid SS \mid \varepsilon$$

generates the language $E = \{w \in \Sigma^* \mid n_0(w) = n_1(w)\}$. Here is another grammar[3] for E:

$$G' : \quad S \to 0P1S \mid 1N0S \mid \varepsilon, \qquad P \to 0P1P \mid \varepsilon, \qquad N \to 1N0N \mid \varepsilon$$

(i) Fill in the LL(1) table of G'. Justify each cell. Is G' LL(1)?

(ii) As Session 30 did for the word `[n,[]]` in the mini-JSON grammar, run the LL(1) parser of G' on the word 001110: give the table of its steps (stack, remaining input, move), and draw the derivation tree it builds.

(iii) What does question (i) tell you about the ambiguity of G'? Justify your answer.

(iv) Show that G is not LL(1). Does this alone prove that G is ambiguous? Is G ambiguous?

[3] The fragment $P \to 0P1P \mid \varepsilon$ of G' is the grammar $S \to (S)S \mid \varepsilon$ of the well-parenthesized words of Session 30, with P for S, 0 for (and 1 for). The fragment $N \to 1N0N \mid \varepsilon$ is the same, with the roles of 0 and 1 swapped. You don't have to prove that G' generates E.

Exercise 4. Regular expressions are words too! In this exercise, we work over the alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}, \mathtt{+}, \mathtt{*}, (,)\}$. The following grammar G_R generates the regular expressions over $\{\mathtt{a}, \mathtt{b}\}$:[4]

$$R \to R\mathtt{+}C \mid C, \qquad C \to CK \mid K, \qquad K \to K\mathtt{*} \mid A, \qquad A \to (R) \mid \mathtt{a} \mid \mathtt{b}$$

As for the arithmetic expressions of Session 30, the layers encode the precedence: a regular expression (R) is a union of concatenations (C), a concatenation is a sequence of starred atoms (K), and an atom (A) is a letter or a regular expression in parentheses. For instance, `ab*+b` is the union of `ab*` and `b`, and in `ab*`, the star only applies to `b`.

(i) Show that G_R is not LL(1).

(ii) How can we get rid of the left recursion in G_R? Propose a grammar that generates the same language and has no left recursion.

[4] Without ε and $\emptyset$, to keep things short.